Homework 2 –

Name: Farida Dabit

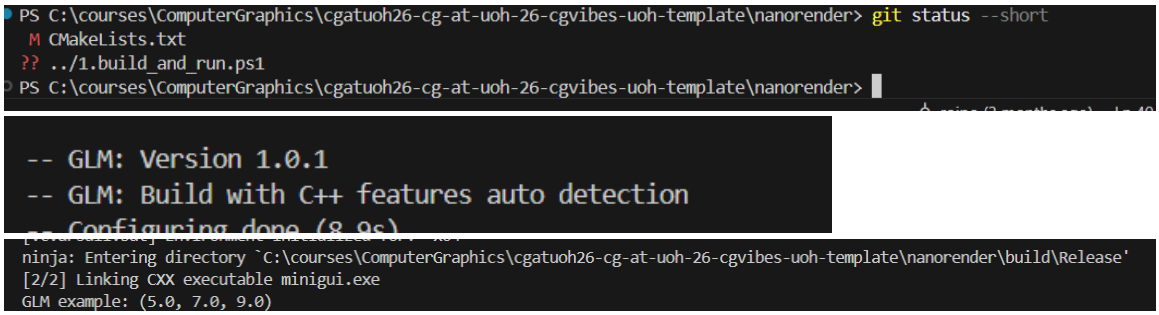
ID: 212693345

Course: Computer Graphics

Part 0 – Introduction to GLM

GLM was added to the project through CMake and linked to the application.
To verify the integration, two glm::vec3 vectors were created and added together.
The program compiled and ran successfully, producing the expected result

(5.0, 7.0, 9.0).



Part 1: Loading and Inspecting 3D Data

For this part, I implemented a function that loads a .obj file and stores its mesh data in memory.

The loader reads:

- **Vertices from lines beginning with v.**
- **Faces from lines beginning with f.**

The vertices are stored as 3D points using glm::vec3, and each triangular face stores the indices of three vertices using glm::ivec3.

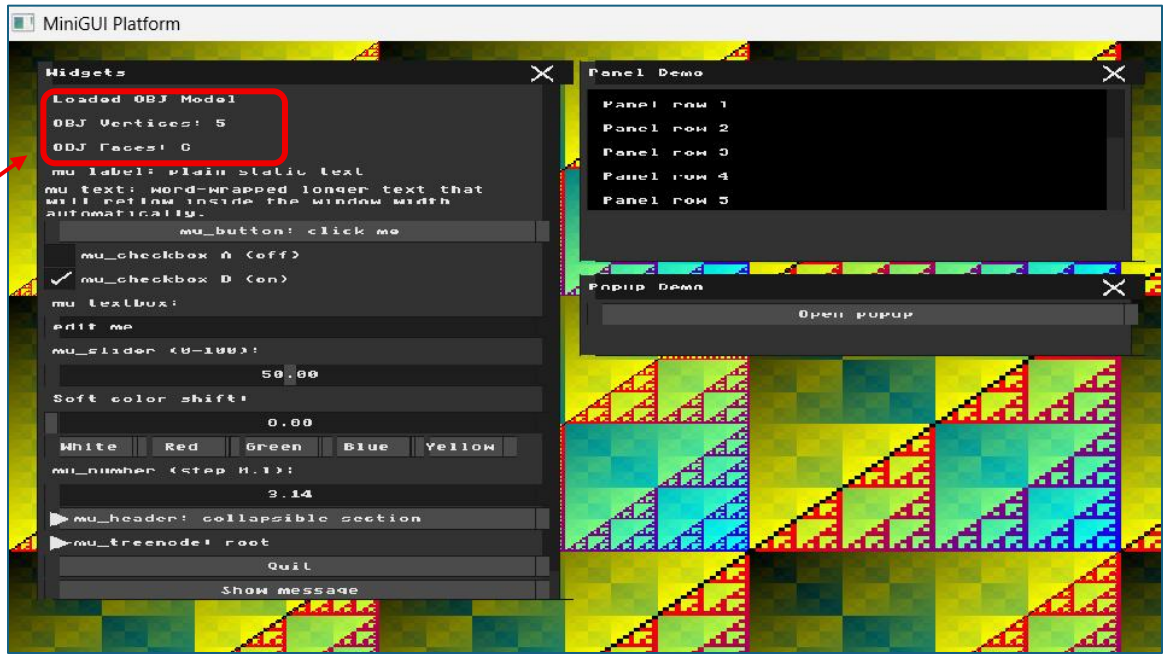
To test the loader, I created a small .obj file containing a pyramid with 5 vertices and 6 triangular faces.

After loading the file, I displayed the number of loaded vertices and faces in the GUI:

- **OBJ Vertices: 5**
- **OBJ Faces: 6**

These values match the contents of the test .obj file, confirming that the model data was loaded correctly.

Figure 1: Loaded OBJ model information showing 5 vertices and 6 faces in the GUI.



Part 2: Normalization and the Viewport Transform

To handle OBJ models with arbitrary coordinate ranges, I first calculated the mesh bounding box by finding the minimum and maximum (x), (y), and (z) values across all loaded vertices.

For the test model, the calculated bounding box was:

- Minimum corner: (-1.0, 0.0, -1.0)
- Maximum corner: (1.0, 1.0, 1.0)

The model size was calculated as:

```
model_size = max_corner - min_corner
```

and the center of the bounding box as:

```
model_center = (min_corner + max_corner) / 2
```

For the test model, this produced

- Model size: `(2.0, 1.0, 2.0)`
- Model center: `(0.0, 0.5, 0.0)`

To fit the model comfortably inside the 1600 × 1200 window, I used 80% of the available width and height. A uniform scale factor was chosen so that the model keeps its original proportions:

```
scale_x = (0.8 * WIDTH) / model_size.x

scale_y = (0.8 * HEIGHT) / model_size.y
```

A single uniform scale factor was then selected using:

```
uniform_scale = min(scale_x, scale_y)
```

Using the same scale factor for all axes preserves the original proportions of the model and prevents distortion.

For the test model, the resulting uniform scale was:

```
uniform_scale = 640
```

The center of the window is:

```
window_center = (WIDTH / 2, HEIGHT / 2, 0)
```

The translation vector was calculated as:

```
translation = window_center - model_center * uniform_scale
```

For the test model, this resulted in:

```
translation = (800, 280, 0)
```

Finally, each transformed vertex is calculated and stored using:

transformed = vertex * uniform_scale + translation

The resulting vertex is then added to the transformed_vertices vector. The original OBJ vertex coordinates remain unchanged.

Result: The bounding box, model dimensions, uniform scale, and translation vector were calculated successfully, allowing the loaded model to be scaled and centered within the framebuffer.

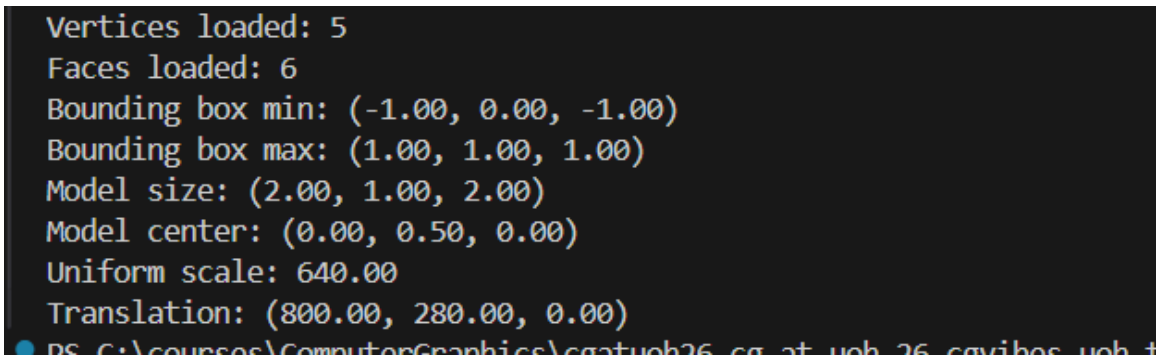


Figure 2: Console output showing the calculated bounding box, model size, model center, uniform scale, and translation.

Part 3 – Orthographic Projection and Wireframe Rendering

In this part, I implemented orthographic projection and wireframe rendering for the loaded 3D mesh.

The transformed_vertices calculated in Part 2 are used for rendering. For each triangular face in the mesh, the three corresponding vertices are retrieved. The orthographic projection is performed by using only the x and y coordinates of each vertex and ignoring the z coordinate:

(x, y, z) → (x, y)

Each triangle is then rendered by drawing its three edges using the draw_line function implemented in HW1:

- v0 → v1
- v1 → v2
- v2 → v0

A separate OBJ test model containing 8 vertices and 12 triangular faces was used so that the wireframe structure could be clearly seen after projection. The model was scaled and centered inside the viewport using the normalization calculated in Part 2.

The result is a static wireframe representation of the 3D mesh. Since hidden surface removal and the Z-buffer are not implemented at this stage, edges from both the front and back triangles may be visible.

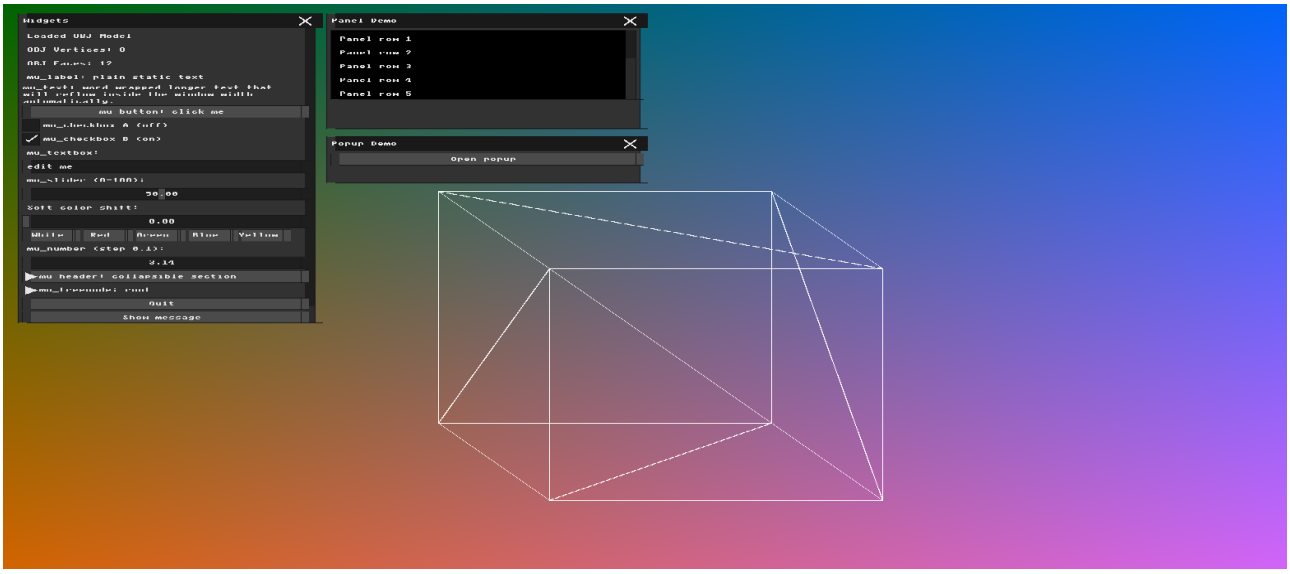


Figure 3: Orthographic wireframe rendering of the test mesh.

Part 4 – Transformation Matrices & Immediate Mode GUI

In this part, I extended the wireframe renderer with interactive controls for 3D transformations.

Using GLM, I created 4×4 transformation matrices for translation, rotation, and scaling. Separate transformation parameters were added for both **Local** and **World** transformations.

The GUI contains X, Y, and Z controls for:

- Local Translation
- Local Rotation
- Local Scale
- World Translation
- World Rotation
- World Scale

Local transformations are applied relative to the model, while world transformations are applied relative to the world coordinate system. Rotation values entered in the GUI are converted from degrees to radians before constructing the rotation matrices.

The transformation parameters are combined into a final 4×4 transformation matrix used by the rendering pipeline before the existing normalization and orthographic projection steps. Changing the GUI values therefore updates the wireframe model interactively.

The screenshot below shows the transformation controls and the resulting transformed wireframe model.

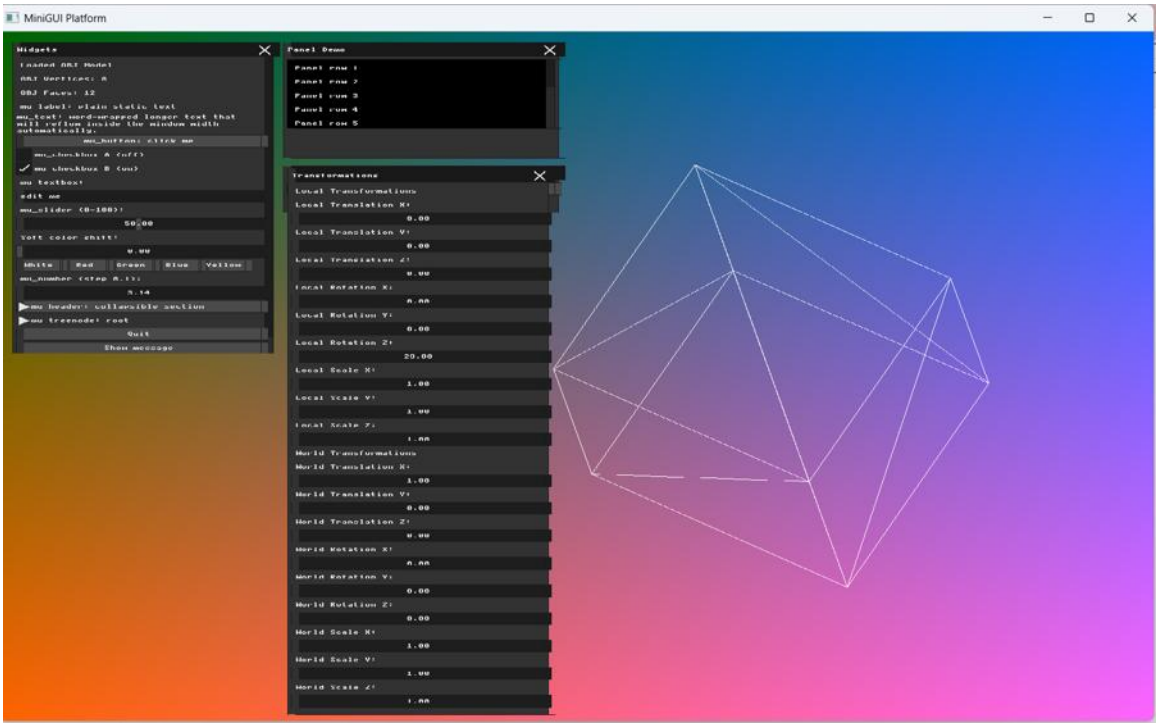


Figure 4: Local and World transformation controls with an interactively transformed wireframe model.

Part 5 – Applying Transformations

In this part, the transformation matrices computed from the GUI values were applied to the mesh vertices before the orthographic projection and wireframe rendering.

Since matrix multiplication is not commutative, changing the order and reference frame of the transformations produces different visual results.

The first configuration uses a translation in the local model frame followed by a rotation in the world frame. The model is translated in its local frame and the resulting position is then affected by the world rotation.

Configuration 1:

- Local Translation X = 1
- World Rotation Z = 45°
- All other translations and rotations = 0
- All scale values = 1

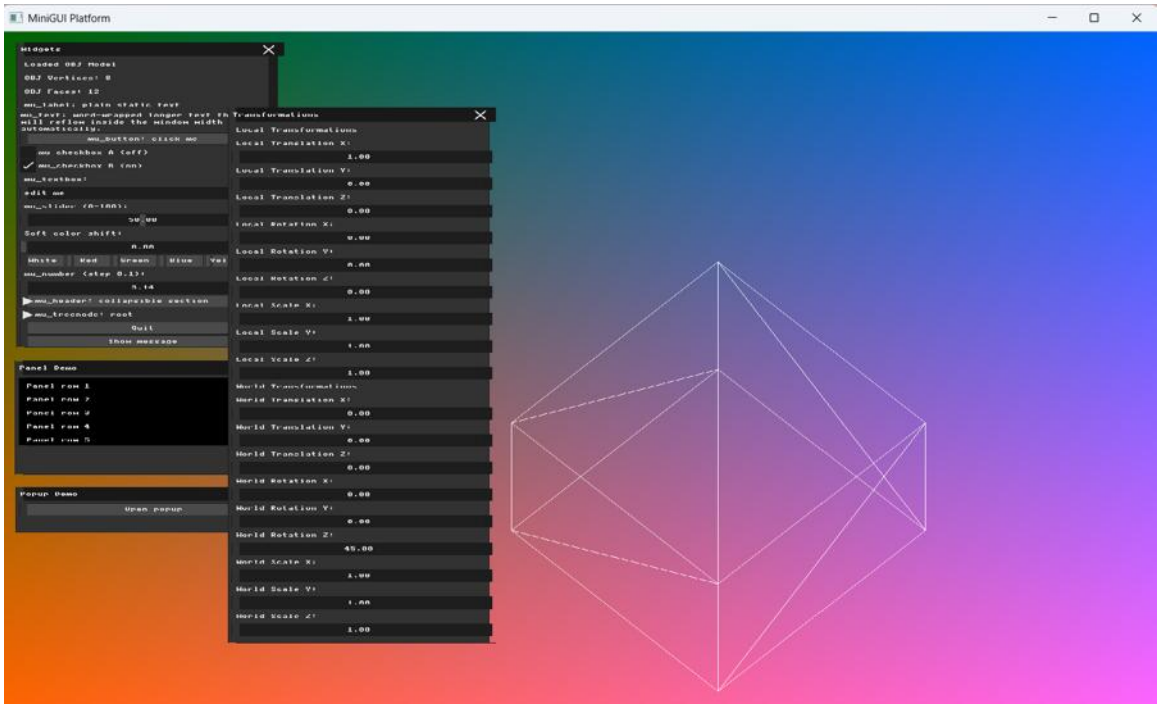


Figure 5: Configuration 1 – Local Translation X = 1 and World Rotation Z = 45°.

The second configuration uses a rotation in the local model frame together with a translation in the world frame. The local rotation is performed around the model center, so the object rotates around itself and is then positioned in the world.

Configuration 2:

- Local Rotation Z = 45°
- World Translation X = 1
- All other translations and rotations = 0
- All scale values = 1

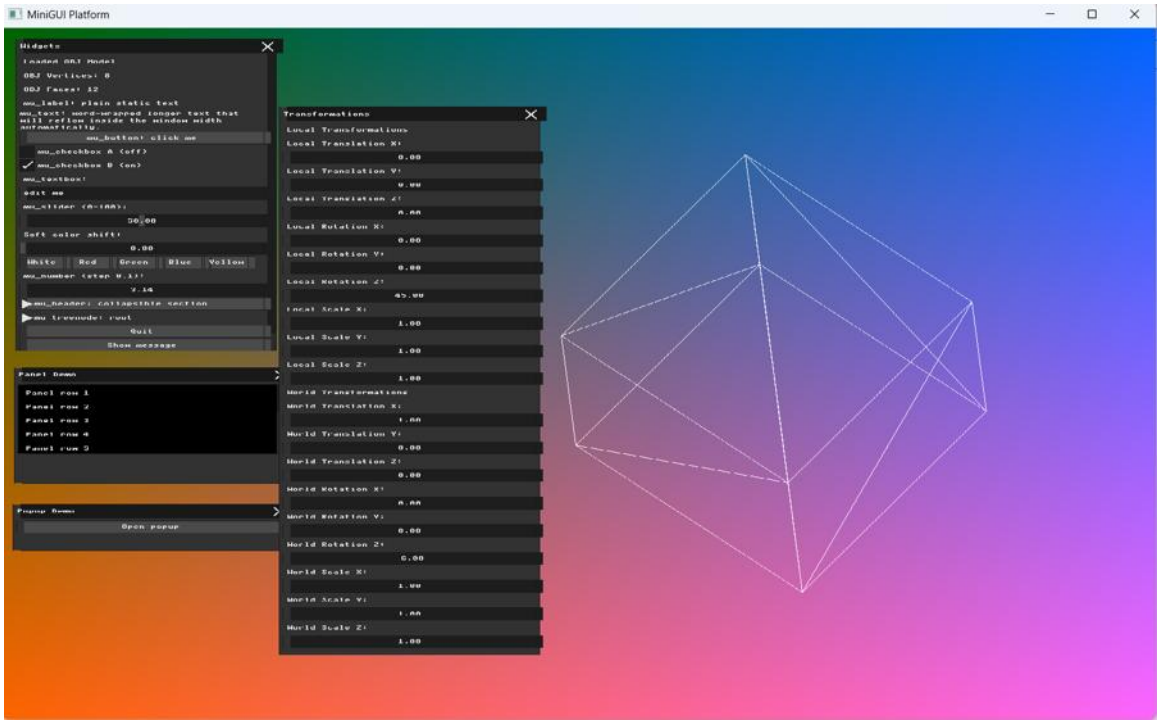


Figure 6: Configuration 2 – Local Rotation Z = 45° and World Translation X = 1.

The two results demonstrate that the order and reference frame of transformations affect the final position and orientation of the model, illustrating that matrix multiplication is not commutative.

Part 6 – Interactive Input Modifiers

For direct interaction, I mapped the keyboard arrow keys to World Translation.

The right and left arrow keys modify the X component of world_translation, while the up and down arrow keys modify the Y component. The keyboard state is read using mfb_get_key_buffer(), and the arrow keys update the corresponding world_translation components.

Because the arrow keys modify the same world_translation variables used by the transformation matrix and the GUI controls, the model moves immediately on screen and the corresponding World Translation values in the GUI are updated at the same time.

Keyboard mapping:

- Right Arrow → increase world_translation.x
- Left Arrow → decrease world_translation.x
- Up Arrow → increase world_translation.y
- Down Arrow → decrease world_translation.y

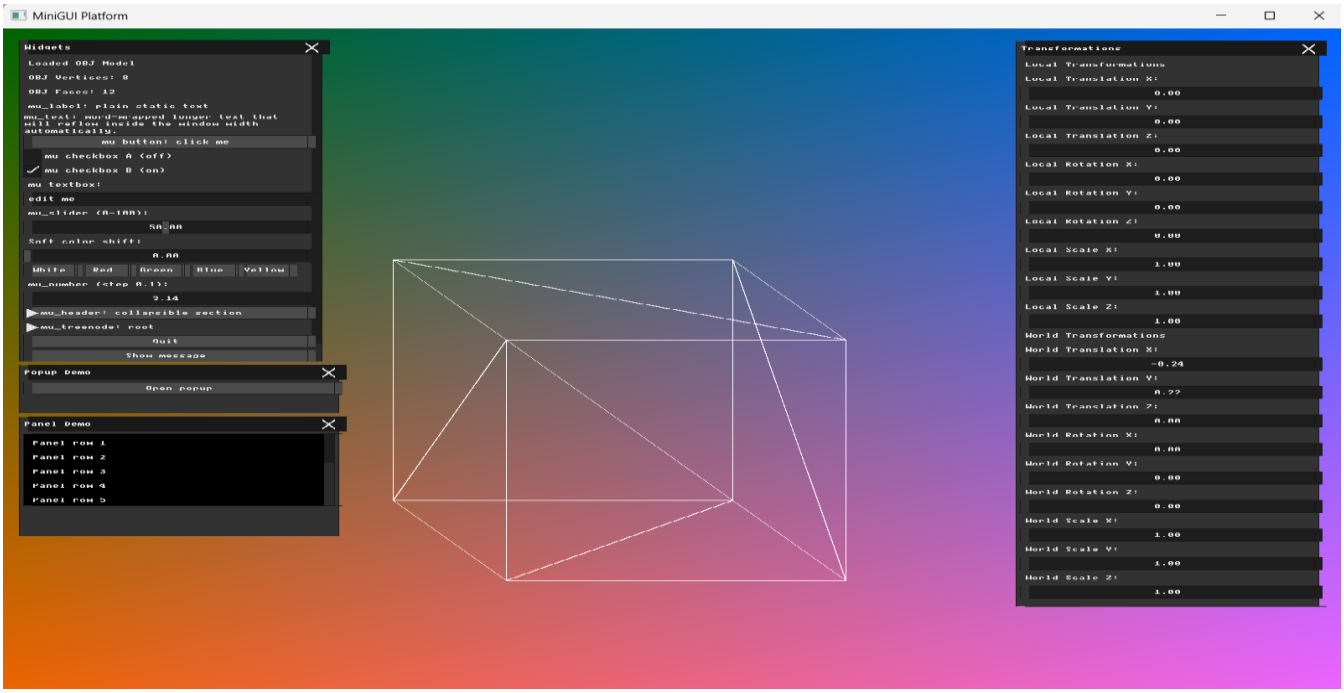


Figure 7: Interactive World Translation using the keyboard arrow keys. The updated X and Y translation values are shown in the Transformations GUI.